ShipSmart — Compare Section Implementation Prompt

Goal

Build a **Compare Section** that loads automatically alongside search results (not on button click) and helps users make a confident shipping decision through a priority-driven "decision cockpit" — not a static spec sheet. The section must match the exact visual and interaction spec below, with a working backend that integrates our existing OpenAI and Gemini LLM clients.

Context

- **Stack**: FastAPI backend (Python), React frontend, existing LLM clients for OpenAI and Gemini.
- **Current state**: The compare section today only renders when the user clicks a "Compare" button in the results list. We are replacing this.
- **New behavior**: When the user submits shipment details and search results load, the compare section must render **below the search results** with 3 default carrier options pre-selected. The user can remove any option (minimum 2) or add another from the full results (maximum 3). When the option set changes, the compare section re-fetches and re-renders.

What this section IS — the design philosophy

Read this before writing a line of code. It will save you from building the wrong thing.

A generic comparison table is a **decision tax**, not a decision aid — it relocates the thinking from the search results page to the compare page and forces the user to hold 7 variables in their head. We are explicitly not building that.

This section is a **decision cockpit**. It has a point of view. It recommends ONE option, names the second and third as alternatives with specific reasons to pick them, and lets the user pivot the entire recommendation around a single control: *"What matters most for this shipment?"*

The rule that separates this feature from AI-slop: if you remove the LLM, the feature should break. The LLM is not decorative. It is load-bearing for the verdict copy, the contextual card tags, and the tradeoff explanations. Prices, ETAs, insurance caps, and historical reliability all come from deterministic data. The LLM's job is the *reasoning over that data* in the user's decision language.

Layer architecture (top to bottom)

1. **Context strip** — one line of muted text summarizing the shipment (item, route, deadline, N of M options).
2. **Priority selector** — 4 pills: *On-time delivery* (default), *Damage protection*, *Lowest price*, *Earliest arrival*. This is the first interactive element. Tapping a pill drives the rest of the section.
3. **Verdict card** — muted surface background. Three lines: small eyebrow ("The pick"), small purpose line ("Best for on-time delivery"), big pick name ("UPS 2nd Day Air"), one-sentence reason that includes both evidence AND concession.
4. **Three option cards** — horizontal grid. Each card has: carrier name, price (big), arrival date, and a short factual tag that updates with the selected priority. The winner card has a 2px teal top border, a "Recommended" pill floating above its top edge, and a matching teal dot in the tradeoffs row.
5. **Add/remove controls** — each card has a small × button top-right (disabled when only 2 cards remain). When fewer than 3 cards are present, a dashed "+ Add carrier" tile appears as the third slot; clicking it opens a picker showing the remaining carriers from the search results.
6. **Tradeoffs section** — labeled "The tradeoffs" (not "deal-breakers"). One row per option. **Even the winner has a tradeoff** — naming the cost of the recommendation is what makes this trustworthy.
7. **Reliability + 100-dots** — labeled "Across 100 shipments on this lane in [month]". Each carrier gets a row with a numeric summary ("96 on-time · 3 late · 1 lost or damaged") and a tight grid of 100 small squares below it, colored green/amber/red. Three-item legend underneath.

Visual spec — exact values, no improvisation

Typography

- Base font: inherit (sans-serif, whatever the app uses).
- Weights: only `400` and `500`. Never `600` or `700`.
- Verdict pick name: `22px / 500`, letter-spacing `-0.01em`.
- Card price: `28px / 500`, letter-spacing `-0.01em`.
- Eyebrow labels ("The pick", "The tradeoffs", section headers): `11px / 500`, letter-spacing `0.04em`, color `var(--color-text-tertiary)`. **Sentence case, no ALL CAPS.**

- Body/tag text: `12-14px / 400`, line-height `1.5-1.55`.
- Context strip: `12px`, color `var(--color-text-tertiary)`.

Colors

Use CSS variables for text/borders/backgrounds so it adapts to light/dark mode:

- `--color-text-primary`, `--color-text-secondary`, `--color-text-tertiary`
- `--color-background-primary`, `--color-background-secondary`
- `--color-border-tertiary` (default 0.5px borders)

Fixed accent colors (hex) — these encode meaning and must not drift:

- Winner teal accent: `#1D9E75` (teal 600 — used for the 2px top border, the dot in tradeoffs row, the "Recommended" pill text color `#0F6E56` on background `#E1F5EE`)
- 100-dots: `#5DCAA5` (on-time green), `#EF9F27` (late amber), `#E24B4A` (damaged/lost red)

Spacing & radii

- Card padding: `1.2rem 1rem 1rem` (top is `calc(1.2rem - 1.5px)` when the card is the winner, to absorb the 2px top border).
- Card gap in grid: `12px`.
- Section vertical rhythm: `2rem` between sections, `1.5rem` between closely-related blocks.
- Card border radius: `var(--border-radius-lg)` (12px).
- Pills border radius: `999px`.
- Dots border radius: `1px`, `aspect-ratio: 1`, gap `1.5px`.

Pill selector

- Inactive: transparent background, `0.5px solid var(--color-border-tertiary)`, `color: var(--color-text-secondary)`.
- Hover: border `var(--color-border-secondary)`, color `var(--color-text-primary)`.
- Active: background `var(--color-text-primary)`, color `var(--color-background-primary)`, border matches background.
- Padding `7px 14px`, font `13px`, transition `all 0.15s ease`.

Cards grid

- `display: grid; grid-template-columns: repeat(3, minmax(0, 1fr)); gap: 12px;`
- Each card: white background (`var(--color-background-primary)`), `0.5px solid var(--color-border-tertiary)` border, `border-radius: var(--border-radius-lg)`.
- Winner card adds `border-top: 2px solid #1D9E75` and reduces `padding-top` by `1.5px` to stay aligned.
- "Recommended" pill: positioned absolute `top: -10px; left: 0.9rem`, background `#E1F5EE`, color `#0F6E56`, `font-size: 10px`, padding `4px 9px`, `border-radius: 999px`, letter-spacing `0.04em`, font-weight `500`.

Remove (×) button

- Position absolute `top: 8px; right: 8px`.
- `20x20px` circle, border `0.5px solid var(--color-border-tertiary)`, transparent background, `font-size: 12px`, color `var(--color-text-tertiary)`.
- Hover: border `var(--color-border-secondary)`, color `var(--color-text-primary)`.
- Disabled (when only 2 cards): `opacity: 0.3`, `cursor: not-allowed`.

Add tile (when fewer than 3 cards)

- Same dimensions as a card but with `border: 0.5px dashed var(--color-border-secondary)`, transparent background.
- Centered `+ Add carrier` label at `13px`, color `var(--color-text-secondary)`.
- On click, shows a popover listing the other carriers from search results (not currently selected), each clickable to add.

100-dots grid

- `display: grid; grid-template-columns: repeat(50, 1fr); gap: 1.5px;`
- `max-width: 560px` so it doesn't dominate on wide screens.
- 100 squares = 2 rows of 50. Each square `aspect-ratio: 1; border-radius: 1px;`.
- Distribution: deterministic shuffle (use a seeded RNG per carrier so renders are stable, not random on every re-render).
- Row structure: header with carrier name (`13px / 500`) on the left and numeric summary (`11px`, `font-variant-numeric: tabular-nums`, color secondary) on the right, dots grid below with `8px` top margin.

Reference code (port this to React, don't redesign it)

The full working HTML/CSS/JS mockup is attached below as the source of truth for every visual detail. Port the markup to JSX, the `<style>` block to a CSS module or styled-components (your choice — match the project's existing pattern), and the `<script>` logic to React state and effects. **Match it pixel-for-pixel.** If you find yourself deviating, stop and re-read the mockup.

```html
```

```
<style>
.sr-only { position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px; ov
.ctx { font-size: 12px; color: var(--color-text-tertiary); margin: 0 0 1.5rem; line-
.prio-q { font-size: 13px; color: var(--color-text-secondary); margin: 0 0 12px; }
.pills { display: flex; gap: 8px; flex-wrap: wrap; margin-bottom: 1.75rem; }
.pill { font-size: 13px; padding: 7px 14px; border-radius: 999px; border: 0.5px soli
.pill:hover { border-color: var(--color-border-secondary); color: var(--color-text-p
.pill.on { background: var(--color-text-primary); color: var(--color-background-prim
.vbox { background: var(--color-background-secondary); border-radius: var(--border-r
.v-eye { font-size: 11px; letter-spacing: 0.04em; color: var(--color-text-tertiary);
.v-purpose { font-size: 13px; color: var(--color-text-secondary); margin: 0 0 4px; }
.v-pick { font-size: 22px; font-weight: 500; margin: 0 0 10px; color: var(--color-te
.v-reason { font-size: 14px; color: var(--color-text-secondary); margin: 0; line-hei
.grid { display: grid; grid-template-columns: repeat(3, minmax(0, 1fr)); gap: 12px;
.card { background: var(--color-background-primary); border: 0.5px solid var(--color
.card.win { border-top: 2px solid #1D9E75; padding-top: calc(1.2rem - 1.5px); }
.rec { position: absolute; top: -10px; left: 0.9rem; background: #E1F5EE; color: #0F
.carrier { font-size: 13px; color: var(--color-text-secondary); margin: 0 0 4px; }
.price { font-size: 28px; font-weight: 500; margin: 0; color: var(--color-text-prima
.date { font-size: 13px; color: var(--color-text-primary); margin: 8px 0 12px; }
.tag { font-size: 12px; color: var(--color-text-secondary); line-height: 1.5; border
.sec { margin-top: 2rem; }
.sec-label { font-size: 11px; letter-spacing: 0.04em; color: var(--color-text-tertia
.to-row { display: grid; grid-template-columns: 130px minmax(0, 1fr); gap: 16px; pad
.to-row:last-child { border-bottom: none; }
.to-carrier { color: var(--color-text-secondary); display: flex; align-items: center
.to-carrier .dot-mark { width: 6px; height: 6px; border-radius: 50%; background: #1D
.to-row.win-row .dot-mark { opacity: 1; }
.to-text { color: var(--color-text-primary); }
.rel-row { padding: 12px 0; border-bottom: 0.5px solid var(--color-border-tertiary);
.rel-row:last-child { border-bottom: none; }
.rel-head { display: flex; align-items: baseline; justify-content: space-between; ga
.rel-name { font-size: 13px; color: var(--color-text-primary); font-weight: 500; }
.rel-nums { font-size: 11px; color: var(--color-text-secondary); font-variant-numeri
.dots { display: grid; grid-template-columns: repeat(50, 1fr); gap: 1.5px; max-width
.d { aspect-ratio: 1; border-radius: 1px; }
.g { background: #5DCAA5; } .y { background: #EF9F27; } .r { background: #E24B4A; }
.legend { display: flex; gap: 16px; font-size: 11px; color: var(--color-text-tertiar
.sw { display: inline-block; width: 8px; height: 8px; border-radius: 1px; vertical-a
</style>
```

Interaction spec

1. Priority pill click

- Updates local React state: `selectedPriority`.
- Re-renders verdict card, card tags, tradeoffs rows, and the winner indicator (teal border, Recommended pill, teal dot in tradeoff row).
- **Does not re-call the backend** — the response from the initial compare request includes all 4 priority scenarios precomputed. The frontend switches between them locally. (Rationale: LLM latency + cost. One compare call returns all 4 scenarios for the 3 selected options.)

2. Remove card

- Fires a new compare request with the reduced option set.
- Shows a loading skeleton in place of the verdict/cards while the request is in flight.
- If remaining count drops to 2, the × buttons become disabled and the add tile appears as the third slot.

3. Add card

- Click the "+ Add carrier" tile → popover with remaining carriers from search results (those not already in the compare set).
- Selecting one fires a new compare request with the expanded set.

4. Search submit

- When the user submits shipment details, results and compare section load together. Compare section picks its 3 defaults automatically (see default selection logic below) and fires the compare request in parallel with the search results render.

Default selection logic (the 3 starting options)

Given the full list of carrier options returned by the search, pick the initial 3 by this priority:

1. **Best public carrier** — the USPS option with the lowest cost that meets the user's deadline. If no USPS option meets the deadline, use the cheapest USPS option regardless.
2. **Best private carrier** — from UPS/FedEx/DHL, the option with the highest on-time reliability on this lane that meets the deadline. Break ties by lower cost.

3. **Cheapest overall** — the absolute lowest-cost option from the full result set, regardless of carrier, as long as it's not already in slots 1 or 2. If it duplicates, fall back to the second-cheapest.

If the search returns fewer than 3 options, pre-select all of them and disable the add tile.

Implement this logic in the backend — the frontend just calls `/compare` with the 3 option IDs.

Backend — API contract

Endpoint

`POST /api/compare`

Request body

```json
{
  "shipment": {
    "item_description": "ceramic vase",
    "declared_value_usd": 200,
    "is_fragile": true,
    "origin_zip": "30301",
    "destination_zip": "98101",
    "deadline_date": "2025-12-19",
    "weight_lb": 3.2,
    "dimensions_in": { "l": 12, "w": 8, "h": 8 }
  },
  "option_ids": ["ups_2da_xxx", "fedex_2d_yyy", "usps_pri_zzz"]
}
```

Response body

```json
```

```
{
  "shipment_summary": "$200 ceramic vase · Atlanta → Seattle 98101 · must arrive by
  "options": [
    {
      "id": "ups_2da_xxx",
      "carrier": "UPS",
      "service_name": "UPS 2nd Day Air",
      "carrier_type": "private",
      "price_usd": 22.00,
      "arrival_date": "2025-12-19",
      "arrival_label": "Fri, Dec 19",
      "insurance_cap_usd": 200,
      "on_time_rate_lane_month": 0.96,
      "claim_approval_rate": 0.94,
      "lane_outcomes_100": { "on_time": 96, "late": 3, "lost_or_damaged": 1 }
    }
  ],
  "scenarios": {
    "ontime": {
      "winner_id": "ups_2da_xxx",
      "verdict": {
        "purpose": "Best for on-time delivery",
        "pick_name": "UPS 2nd Day Air",
        "reason": "96% on-time on this lane in December. $4 more than FedEx, but you
      },
      "card_tags": {
        "ups_2da_xxx": "96% on-time · full $200 covered",
        "fedex_2d_yyy": "1 day earlier · $100 insurance cap",
        "usps_pri_zzz": "$8 cheaper · no guaranteed date"
      },
      "tradeoffs": [
        { "option_id": "ups_2da_xxx", "text": "$4 more than FedEx. Arrives one day l
        { "option_id": "fedex_2d_yyy", "text": "$100 of your vase stays uninsured. C
        { "option_id": "usps_pri_zzz", "text": "No guaranteed date. 22 out of 100 sh
      ]
    },
    "damage":  { /* same shape */ },
    "price":   { /* same shape */ },
    "speed":   { /* same shape */ }
  }
}
```

All 4 scenarios are precomputed in one backend call so the frontend can pivot locally with zero latency on pill clicks.

Backend — implementation plan

1. Gather deterministic data

For each `option_id`, pull from existing carrier-rate / lane-history services:

- Price, service name, carrier type (public/private), arrival date
- Insurance cap for this declared value
- Historical on-time rate **for this origin-destination lane in this specific month** (critical — generic rates aren't useful)
- Historical claim approval rate for this carrier
- Last-100-shipments outcomes on this lane: integer counts of on_time / late / lost_or_damaged

If lane-month granularity doesn't exist yet, fall back to lane-annual rates and note the limitation in a code comment — do not fabricate numbers.

2. Build the decision context

Assemble a structured JSON blob that gets passed to the LLM. This is the one input — the LLM doesn't call other tools:

```json
{
  "shipment": { /* full shipment object */ },
  "options": [
    {
      "id": "...", "carrier": "UPS", "service_name": "UPS 2nd Day Air",
      "price": 22, "arrival_days_before_deadline": 0, "arrival_label": "Fri, Dec 19"
      "insurance_cap": 200, "declared_value": 200, "coverage_gap": 0,
      "on_time_rate": 0.96, "claim_approval_rate": 0.94,
      "outcomes_per_100": { "on_time": 96, "late": 3, "damaged_or_lost": 1 }
    }
  ]
}
```

3. LLM call — single prompt, structured output

Use the existing OpenAI client as primary, Gemini as fallback. Request JSON mode / structured output with the schema matching the `scenarios` field above. One call generates all 4 scenarios.

System prompt:

You are the decision engine for ShipSmart's compare section. You help users pick the right shipping option by stating a clear recommendation and naming the real tradeoffs honestly.

Given a shipment and 2-3 carrier options with deterministic data (price, ETA, insurance, lane reliability), produce 4 priority scenarios: ontime, damage, price, speed.

For each scenario:

1. Pick the winner based on that priority, using the data. Do not pick arbitrarily — the data determines it.
2. Write a verdict with three parts:
   - `purpose`: "Best for [priority]" (6 words max)
   - `pick_name`: the service name of the winner
   - `reason`: ONE sentence, 15-25 words, that states BOTH the evidence (why it wins) AND the concession (what it costs). Use the format: "[evidence]. [concession sentence]." Example: "96% on-time on this lane in December. $4 more than FedEx, but your full $200 is covered."
3. Write a short factual tag for each option (4-8 words, middle-dot separator). For the winner: its strongest factual position. For losers: what you'd GAIN by picking them instead — e.g. "1 day earlier · $100 insurance cap", "$8 cheaper · no guaranteed date".
4. Write one tradeoff line per option (12-25 words). EVEN THE WINNER gets a tradeoff — name the cost of picking it. Never say "None" or "No tradeoffs" — every choice has a cost.

Rules for the reason and tradeoff copy:
- Use specific numbers from the data, not vague words like "much" or "significantly".
- Never invent facts not in the data. If claim approval rate isn't provided, don't mention it.
- If two options are effectively tied for a priority (within 3% reliability AND within $3 AND same day), say so in the verdict: "X and Y are effectively equivalent for this shipment. [Pick one with a specific reason — cheaper, better coverage, whatever the tiebreaker is]."
- Never use em-dashes in prose. Use periods.
- Sentence case. No ALL CAPS. No marketing fluff. No "excellent", "fantastic", "perfect", "amazing".
- Write like a surgeon, not an assistant.

Return ONLY the JSON matching the schema. No preamble, no explanation.

Response schema (enforce via OpenAI `response_format` / Gemini JSON mode):

```
json
```

```
{
  "type": "object",
  "required": ["ontime", "damage", "price", "speed"],
  "properties": {
    "ontime": { "$ref": "#/$defs/scenario" },
    "damage": { "$ref": "#/$defs/scenario" },
    "price":  { "$ref": "#/$defs/scenario" },
    "speed":  { "$ref": "#/$defs/scenario" }
  },
  "$defs": {
    "scenario": {
      "type": "object",
      "required": ["winner_id", "verdict", "card_tags", "tradeoffs"],
      "properties": {
        "winner_id": { "type": "string" },
        "verdict": {
          "type": "object",
          "required": ["purpose", "pick_name", "reason"],
          "properties": {
            "purpose": { "type": "string" },
            "pick_name": { "type": "string" },
            "reason": { "type": "string" }
          }
        },
        "card_tags": {
          "type": "object",
          "additionalProperties": { "type": "string" }
        },
        "tradeoffs": {
          "type": "array",
          "items": {
            "type": "object",
            "required": ["option_id", "text"],
            "properties": {
              "option_id": { "type": "string" },
              "text": { "type": "string" }
            }
          }
        }
      }
    }
```

```
  }
}
```

4. Validate & merge

- Validate the LLM output matches the schema. If it fails, retry once with a stricter prompt, then fall back to Gemini. If both fail, return a deterministic minimal scenario object (winner by simple heuristic per priority, generic tradeoff copy) and log the failure.
- Validate that every `winner_id` exists in the option set and that `card_tags` has a key for every option. Reject and retry if not.
- Merge LLM scenarios with the deterministic option data into the final response.

5. Cache

Cache compare responses by `(shipment_hash, sorted_option_ids_hash)` with a 15-minute TTL. Live rates expire quickly but the same user is likely to toggle options within that window.

Frontend — implementation plan

File structure (adapt to existing conventions)

```
src/components/compare/
  CompareSection.tsx          — top-level container, manages state and data fetch
  PrioritySelector.tsx        — the pill row
  VerdictCard.tsx             — the muted-background verdict block
  OptionCard.tsx              — individual carrier card with winner treatment
  AddCarrierTile.tsx          — dashed "+ Add carrier" tile with popover
  TradeoffsList.tsx           — the tradeoffs rows
  LaneReliability.tsx         — the 100-dots visualization
  compare.types.ts            — TypeScript types matching the API contract
  compare.api.ts              — API client function
  CompareSection.module.css   — all the styles (or styled-components if that's the
project pattern)
```

State model (in `CompareSection.tsx`)

```ts
```

```
type CompareState = {
  optionIds: string[];            // 2-3 ids, user-editable
  selectedPriority: 'ontime' | 'damage' | 'price' | 'speed';  // default 'ontime'
  data: CompareResponse | null;  // full response, all 4 scenarios cached
  isLoading: boolean;
  error: string | null;
};
```

Data flow

1. On mount (and whenever `optionIds` changes): call `/api/compare`, set `data`, clear loading.
2. Pill click: just update `selectedPriority`. No network call.
3. Remove click: `setOptionIds(ids.filter(id => id !== removed))`. The effect triggers a re-fetch.
4. Add selection: `setOptionIds([...ids, added])`. Re-fetch.
5. Derive the active scenario with `data?.scenarios[selectedPriority]` and pass it down.

Seeded shuffle for 100-dots

The dot order must be **stable across renders** (don't re-shuffle on every pill click — that would be visual noise). Use a seeded RNG keyed on `option_id` so the same carrier always produces the same dot arrangement:

```ts
function seededShuffle<T>(arr: T[], seed: number): T[] {
  const result = [...arr];
  for (let i = result.length - 1; i > 0; i--) {
    seed = (seed * 9301 + 49297) % 233280;
    const j = Math.floor((seed / 233280) * (i + 1));
    [result[i], result[j]] = [result[j], result[i]];
  }
  return result;
}
```

Seed = hash of `option_id`.

Accessibility

- Priority pills: render as `<button role="radio">` inside `<div role="radiogroup" aria-label="What matters most for this shipment">`. The active pill has `aria-

`checked="true"`.

- Winner card: add `aria-label="Recommended option: [carrier service name]"` and an `sr-only` `<span>` announcing the verdict purpose when the priority changes.
- Remove button: `aria-label="Remove [carrier name] from comparison"`.
- 100-dots: wrap each row in a `<figure>` with a `<figcaption class="sr-only">` like `"UPS 2nd Day Air: 96 on-time, 3 late, 1 lost or damaged out of 100 shipments on this lane in December"`. The dots themselves are decorative (`aria-hidden="true"`).
- Announce priority changes with a visually-hidden `aria-live="polite"` region containing the new verdict purpose.

Integration point with search results

Find the existing search results component. Immediately after it renders, mount `<CompareSection shipment={shipment} searchResults={results} />`. The CompareSection's own effect runs the default-selection logic against `searchResults` on mount.

Acceptance criteria

A reviewer should be able to tick all of these:

- [ ] Compare section appears **automatically** below search results — no Compare button required.
- [ ] Three default options are pre-selected using the public/private/cheapest logic.
- [ ] All 4 pills work and update: verdict, card tags, winner indicators, and tradeoff copy — with zero network latency after the initial load.
- [ ] The winner card has a 2px teal top border (`#1D9E75`), a "Recommended" pill positioned at `top: -10px`, and a matching teal dot next to its name in the tradeoffs row.
- [ ] **Even the winner has a tradeoff line** — it is never "None" or empty.
- [ ] Removing a card triggers a fresh compare request and disables further removal when only 2 remain.
- [ ] The add tile appears only when fewer than 3 cards are present and shows a popover of the other available carriers from the search results.
- [ ] 100-dots row for each carrier: numeric summary above, 2×50 grid of colored squares below, max-width 560px, dot order stable across re-renders (seeded shuffle).
- [ ] All typography uses weights 400 and 500 only. No 600 or 700.
- [ ] All colors use CSS variables except the 4 fixed accent hex values (`#1D9E75`, `#5DCAA5`, `#EF9F27`, `#E24B4A`, plus `#E1F5EE`/`#0F6E56` for the Recommended pill).
- [ ] LLM call returns structured JSON matching the schema. On validation failure, retries once, then falls back to Gemini, then to a deterministic minimal response.

- [ ] LLM response is cached by `(shipment_hash, option_ids_hash)` for 15 minutes.
- [ ] No marketing words in generated copy ("excellent", "amazing", etc). If you see one in a real response, tighten the prompt.
- [ ] Section gracefully handles: fewer than 3 search results, LLM failure, stale cache, one option with missing lane-history data.

Things NOT to do (common failure modes)

- Do not build a generic feature-comparison table with checkmark rows. The grid is opinionated, not symmetric.
- Do not use the LLM to "summarize" the options. The LLM's job is the verdict, tags, and tradeoffs — not a paragraph about each carrier.
- Do not invent reliability data. If the lane-history service doesn't have the right granularity, use what's available and mark the gap in a code comment.
- Do not make pill clicks re-call the backend. All 4 scenarios come back in one response.
- Do not add icons, gradients, shadows, or extra chrome. The design is flat on purpose.
- Do not use ALL CAPS for any labels, including section headers.
- Do not let the winner card claim "no tradeoffs." Every choice costs something. Name it.
- Do not fetch carrier logos or brand imagery. Text only for carrier names.

What to ship in the first PR

1. Backend: `/api/compare` endpoint, LLM client integration, validation, caching, default-selection logic.
2. Frontend: all 7 components listed above, wired into the search results page, with pixel-matched styling.
3. Tests: one integration test for the endpoint with a fixed shipment and 3 options, snapshot-asserting the response shape (not the exact LLM copy, which is non-deterministic — assert schema validity and that all 4 scenarios are present with valid `winner_id` values).
4. One Storybook story per component if the project uses Storybook, with `CompareSection` getting a top-level story using mocked API data for all 4 priority states.

Ship this as a single PR titled `feat(compare): decision-cockpit compare section with LLM-driven priority pivoting`.